

Design Note — LLD Practice Platform

1. Product Goal

The LLD Practice Platform is designed around one simple learning loop:

Choose a problem → Design → Submit → Get feedback → Review → Try again

The goal is not to tell a learner that their design is simply “right” or “wrong”. Low-Level Design is open-ended: different class structures can solve the same requirements. The product therefore focuses on explaining observable issues, requirement coverage, and contextual design considerations.

The MVP is intentionally small. It supports a few LLD problems, structured design submissions, deterministic checks, heuristic observations, optional AI feedback, attempt history, and retrying previous problems.

2. MVP and User Flow

Problem Selection

The learner starts on the Problems page and sees a small set of design problems such as:

- Design a Parking Lot
- Design a Vending Machine
- Design an Elevator System

Problems contain requirements, constraints, difficulty, and expected concepts. Expected concepts are treated as useful concepts to consider, not as mandatory answer keys.

Practice / Design

The learner opens a problem and creates an attempt. The MVP uses a structured submission containing:

- ****Classes**** — class names
- ****Responsibilities**** — what each class owns or does
- ****Relationships**** — how classes interact
- ****Patterns Used**** — patterns the learner chose
- ****Optional Code Stub**** — additional implementation context

The learner can save a draft or submit the attempt.

Evaluation

Submission moves through a controlled lifecycle:

``draft → submitted → evaluating → evaluated``

A total pipeline failure can move an attempt to ``failed``. An AI-provider failure does not make the whole attempt fail; deterministic and heuristic feedback can still be returned.

Feedback

The report is divided by confidence:

1. Structural Checks — High confidence
2. Worth Considering — Medium confidence
3. Requirement Coverage
4. AI Insights — Contextual

This separation is important because an AI suggestion should not look like an objective compiler-style error.

History and Retry

The learner can view previous attempts, open detailed feedback, delete an attempt, and use Try Again to return to the same problem with a fresh attempt.

3. Backend Architecture

The application uses a practical MERN-style monolith:

React → Express/Node.js → MongoDB

The backend is divided into controllers, services, models, and evaluators rather than putting all logic inside route handlers.

Main Backend Components

Component	Responsibility
Problem model	Stores LLD problem requirements and constraints
Attempt model	Stores learner submissions, status, and feedback
AttemptService	Coordinates draft, submit, evaluation, and status transitions
RuleEvaluator	Produces deterministic structural findings
Requirement Coverage	Checks whether important requirement concepts appear in the submission
LLMEvaluator	Uses Gemini for contextual design feedback
CompositeEvaluator	Combines deterministic and AI evaluation
Controllers	Expose HTTP API endpoints
MongoDB	Persists problems and attempts

The main API surface is:

- `GET /api/problems`
- `GET /api/problems/:id`
- `POST /api/learners`
- `POST /api/attempts`
- `PATCH /api/attempts/:id`
- `POST /api/attempts/:id/submit`
- `GET /api/attempts/:id`
- `GET /api/attempts?learnerId=...`
- `DELETE /api/attempts/:id`

4. Domain Model

Problem

A problem contains:

- `title`
- `difficulty`
- `requirements`

- ``constraints``
- ``expectedConcepts``

``expectedConcepts`` are not treated as a canonical solution. They are signals that can help the evaluator and learner think about relevant design concepts.

Attempt

An attempt belongs to a learner and a problem.

It stores:

- ``learnerId``
- ``problemId``
- ``submission``
- ``status``
- ``feedback``
- timestamps

The submission contains classes, responsibilities, relationships, patterns, and an optional code stub.

FeedbackReport

Feedback is stored separately inside the attempt as structured categories:

- structural findings
- heuristic findings
- requirement coverage
- AI insights
- AI availability
- summary

This makes feedback easy for the frontend to render and allows additional evaluation layers later.

5. Evaluation Design

The most important architectural decision is that the platform does not rely on the LLM alone.

Layer 1 — Structural Evaluation

These checks are deterministic and high-confidence.

Examples:

- class has a name
- class has responsibilities
- relationships do not obviously reference undeclared classes
- submission has the expected structure

A deterministic evaluator can be tested consistently and gives the learner feedback that does not depend on model output.

Layer 2 — Heuristic Evaluation

Heuristics look for useful signals, such as concepts appearing in the learner's submission.

These findings are deliberately phrased as observations rather than verdicts. For example, the system may say that a concept is worth considering instead of claiming that a specific design pattern is required.

Layer 3 — AI Evaluation

Gemini receives:

- the original problem
- the learner's complete structured submission
- deterministic findings

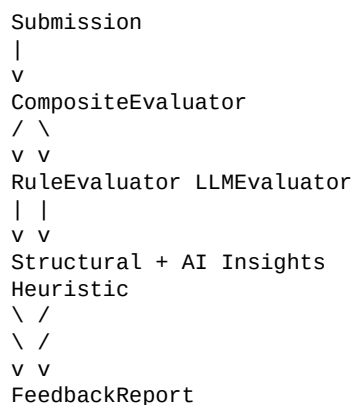
The model is asked to reason about areas such as:

- abstraction
- coupling
- cohesion
- responsibility placement
- requirement coverage
- extensibility
- trade-offs
- appropriateness of patterns used

The AI result is kept separate from deterministic evidence so the learner can distinguish machine-checkable findings from contextual suggestions.

Composite Evaluation

Conceptually:

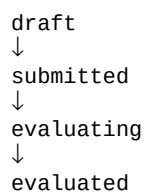


The deterministic findings are also passed to the AI evaluator as context. This reduces duplicated analysis and gives the model concrete evidence to consider.

6. Failure and Evaluation States

The platform treats evaluation failure as a normal engineering concern.

Normal flow



AI failure

If Gemini is unavailable, times out, or returns an unusable response:

submitted → evaluating → evaluated

The attempt is still considered evaluated because structural and heuristic feedback remain available. The feedback explicitly records:

`llmAvailable: false`

Total evaluation failure

If the core evaluation pipeline itself fails unexpectedly, the attempt can enter:

`failed`

This distinction prevents a temporary external AI problem from making the whole learning experience unusable.

7. Why These Design Choices

Structured submission instead of completely free-form text

A free-form answer is natural for a human but difficult to evaluate consistently in a small MVP. Structured classes and responsibilities provide stable machine-readable signals.

An optional code stub keeps the product flexible without requiring code execution, compilation, or a sandbox.

Hybrid evaluator instead of LLM-only evaluation

An LLM is useful for contextual reasoning, but deterministic checks are more predictable and explainable for objective structural issues.

Keeping both layers separate also makes testing easier.

Monolith instead of microservices

The product does not currently need independent deployment, distributed queues, service discovery, or event-driven infrastructure. A modular monolith keeps the code understandable and leaves room to extract components later if scale actually requires it.

Explicit status transitions instead of a State Pattern

The number of attempt states is small. An allowed-transition map inside the service is easier to understand and maintain than introducing a full State Pattern for the MVP.

Confidence-aware feedback

Not every finding has the same certainty. Showing confidence categories makes the product more honest and helps learners treat AI suggestions as guidance rather than unquestionable truth.

8. Extensibility

The evaluation flow is designed around abstractions rather than a single hard-coded evaluator.

A submission can conceptually expose:

```
getClasses()  
getRawText()
```

This makes it possible to add other submission types later, such as:

- diagram-based submissions
- richer code submissions
- text explanations
- UML representations

Similarly, evaluators follow a common contract:

```
evaluate(submission, problem) → EvaluationResult
```

This allows additional evaluators to be introduced without changing the core attempt workflow.

Future versions could add a diagram evaluator, a stronger semantic requirement evaluator, or a code-analysis evaluator.

9. Frontend Design

The frontend is a React application with a focused developer-learning interface.

Main screens:

- **Problems** — browse and filter challenges
- **Practice** — create and save a design
- **Evaluation Report** — review structured feedback
- **Attempt History** — review previous work
- **Attempt Detail** — inspect a specific submission
- **Learn LLD** — supporting learning material

The feedback page uses vertically stacked sections instead of dense multi-column cards. This is deliberate: AI feedback can contain longer explanations, so readability is more important than fitting everything into a compact grid.

10. Testing and Engineering Quality

The backend includes tests for:

- evaluator behaviour
- requirement coverage
- LLM evaluation paths
- composite evaluation
- attempt submission
- status transitions
- API routes
- delete behaviour
- failure handling

The frontend includes tests for core pages and user interactions, and the production build is checked separately.

GitHub Actions is used as a CI quality gate for backend tests, frontend tests, and the frontend build.

Deployment is handled separately through Render.

11. Trade-offs and Known Limitations

This is intentionally an MVP, so some areas remain simplified.

- Requirement coverage is currently keyword/signal based, so it cannot fully understand semantics.
- LLD quality is subjective; AI feedback can still be imperfect.
- There is no code execution or compilation sandbox.
- Authentication and multi-user authorization are kept minimal for the prototype.

- The problem set is intentionally small.
- AI evaluation depends on an external provider and can be unavailable.

These limitations are explicit rather than hidden. The architecture leaves clear extension points for improving them later.

12. Future Direction

The next meaningful improvements would be:

1. Better semantic requirement coverage.
2. Diagram/UML submission support.
3. More calibrated feedback using manually reviewed examples.
4. Comparison between multiple valid design approaches.
5. Richer trade-off explanations.
6. More LLD problems with varied difficulty.
7. Stronger ownership and authentication for production use.
8. Evaluation analytics to understand where learners repeatedly struggle.

The core design principle would remain the same:

The platform should help the learner reason about design decisions, not simply judge whether an answer matches a predefined solution.